

# Analysis on Implementing Reusable Utility Functions in Playwright

## 1. Introduction

Playwright is one of the widely used automation testing frameworks for web applications. While working on automation scripts, there are several actions that are repeatedly used across different test cases, such as logging into the application, taking screenshots, waiting for elements, uploading files, and handling test data.

Writing the same code multiple times increases the effort required to maintain the automation suite. If any change is needed in the future, it has to be updated in multiple places, which can be time-consuming and error-prone.

To overcome this challenge, reusable utility functions can be implemented. These functions help centralize common functionalities and allow them to be reused across different test scripts. This document provides an analysis of how reusable utility functions can be implemented in Playwright and the benefits they bring to an automation framework.

## 2. What are Reusable Utility Functions?

In automation testing, there are certain actions that are performed repeatedly across multiple test cases. Instead of writing the same code every time repeatedly, these common actions can be placed in separate functions and reused whenever required. Such functions are known as reusable utility functions.

Reusable utility functions help organize the code better and avoid duplication. They also make the framework easier to maintain because any future changes need to be made only once instead of updating all the test scripts we written.

For example, functionalities like user login, taking screenshots, waiting for web elements, file uploads, and test data generation are commonly used in many test cases. Creating utility functions for these actions allows testers to reuse the same logic throughout the framework.

This approach improves code readability, reduces maintenance effort, and helps in building a more scalable automation framework.

Example is given below:

## 1. Login code written directly in script

```
TS example.ts 2 X
TS example.ts > test('Utility Functions') callback
1  import {test, expect} from '@playwright/test'
2
3  test('Utility Functions', async({page})=>{
4    await page.fill('#username', username);
5    await page.fill('#password', password);
6    await page.click('#loginBtn');
7  });
8
```

## 2. Reusable login code written before and directly reusing here in test script

```
await login(page, username, password);
```

# 3. Why Reusable Utility Functions are Needed?

In automation testing projects, many actions are performed repeatedly across different test cases. For example, logging into the application, taking screenshots, waiting for elements, and uploading files are common activities that may be required in multiple scripts.

If the same code is written again and again, the framework becomes difficult to maintain. Whenever there is a change in the application, the same update has to be made in multiple test cases. This not only increases effort but also creates the possibility of missing some updates.

Reusable utility functions help solve this problem by keeping commonly used logic in one place. Instead of writing the same code repeatedly, testers can simply call the required utility function whenever needed.

Some of the key benefits of using reusable utility functions are:

## Reduced Code Duplication

Common functionalities can be written once and reused in multiple test cases, reducing writing the same code again and again.

### **Easier Maintenance**

Any future changes can be made in a single location rather than updating multiple scripts.

### **Better Readability**

Test scripts become cleaner and easier to understand because only the business flow is visible in the test case.

### **Improved Scalability**

As the automation framework grows, reusable utility functions help keep the code organized and manageable.

### **Consistency**

Using the same utility functions across the framework ensures that common operations are performed in a consistent manner.

## **4. Common Reusable Utility Functions in Playwright**

While designing an automation framework, certain functionalities are used frequently across multiple test cases. Instead of implementing the same logic repeatedly, these functionalities can be converted into reusable utility functions.

One of the most commonly used utilities is a **Login Utility**. Since many test cases require users to log into the application, creating a reusable login function helps avoid writing the same steps in every script.

A **Screenshot Utility** is also useful, especially for capturing screenshots during test execution or when a test fails. This helps in debugging and reporting issues.

Another important utility is a **Wait Utility**. Web applications may take time to load elements, and using reusable wait functions helps handle synchronization more effectively.

Similarly, a **File Upload Utility** can be created for applications that require users to upload documents or files. This avoids repeating the same upload logic in multiple test cases.

A **Test Data Utility** can be used to manage test data generation or retrieval. This is helpful when multiple test cases require similar sets of data.

In some projects, a **Navigation Utility** can also be implemented to handle common navigation steps within the application.

The utilities implemented in a framework may vary depending on project requirements, but the overall goal remains the same: reducing duplicate code and improving maintainability.

## 5. Implementation Approach

To implement reusable utility functions in Playwright, commonly used functionalities are identified and moved into separate utility files. These files are usually maintained inside a dedicated **utils** folder.

The utility functions can then be imported and reused across multiple test cases whenever required. This helps reduce code duplication and makes the framework easier to maintain.

A typical approach includes:

- Identifying common functionalities.
- Creating reusable utility functions.
- Storing them in utility files.
- Importing and using them in test scripts.

### Project Structure

```
project
|
├─ tests
├─ pages
├─ utils
|   ├─ loginUtil.ts
|   ├─ screenshotUtil.ts
|   ├─ waitUtil.ts
|   └─ fileUploadUtil.ts
|
├─ test-data
└─ playwright.config.ts
```

## 6. Sample Implementation

The following example shows how a reusable login utility can be created and used in Playwright.

Login Utility:

```
export async function login(page, username, password) {  
    await page.fill('#username', username);  
    await page.fill('#password', password);  
    await page.click('#loginBtn');  
}
```

Usage in Test Script:

```
test('Verify Login', async ({ page }) => {  
    await page.goto('https://example.com');  
    await login(page, 'testuser', 'password');  
});
```

This approach allows the login functionality to be maintained in a single location and reused across multiple test cases.

## 7. Conclusion

Reusable utility functions play an important role in building a maintainable and scalable Playwright automation framework. By moving common functionalities into reusable methods, code duplication can be reduced and test scripts become easier to manage. Implementing utility functions improves readability, simplifies maintenance, and supports the long-term growth of the automation framework.